

Predavanje 4 — Ulazno/izlazne operacije

Programiranje za UNIX

Damir Krstinić Maja Braović

FESB — Sveučilište u Splitu

- ▶ Izvršna datoteka nastaje u dva odvojena koraka: **prevođenje** (`.c` $\rightarrow$ `.o`) i **povezivanje** (`.o` $\rightarrow$ program).
- ▶ gcc bez `-c` radi oboje; s `-c` staje nakon prevođenja. Bez `-o` izlaz se zove `a.out`.
- ▶ Program se pokreće s `./ime`, jer trenutni direktorij nije u PATH-u.
- ▶ Nakon izmjene iznova se prevodi samo promijenjena datoteka, ali se povezivanje ponavlja uvijek.
- ▶ make iz pravila i vremena izmjene sam zaključuje koje korake treba izvesti.
- ▶ Pravilo bez naredbi preusmjerava; pravilo bez ovisnosti izvršava se bezuvjetno.
- ▶ Objektne datoteke pakiraju se u archive alatom `ar`; povezuju se preko `-L` i `-l`.

1. **Deskriptori datoteka** — kako proces referencira otvorene datoteke.
2. **Sistemske pozivi** — open, creat, close, read, write.
3. **Primjeri** — od čitanja datoteke do vlastite inačice cat-a.
4. **Pozicioniranje i prava** — lseek i umask.

1.

Deskriptori datoteka

Što možemo s datotekom?

- ▶ Uobičajeni odgovori: obrisati je, preimenovati, kopirati.
- ▶ Sve je to točno — ali jednako kao kad bismo za stolicu rekli da je možemo prebojati, premjestiti ili nacijepati. Ne otkriva čemu stolica služi: na nju se **sjeda**.
- ▶ Datoteka postoji da bismo u nju nešto **upisali** ili iz nje nešto **pročitali**. To je sve.
- ▶ Uz to nam treba još samo nekoliko funkcionalnosti: **otvaranje**, **zatvaranje** i **pozicioniranje** unutar datoteke.

Svaki operacijski sustav mora osigurati upravo te operacije da bi bio upotrebljiv.

Zašto sistemski pozivi

- ▶ U C-u za rad s datotekama koristimo `fopen`, `fscanf`, `fprintf`, `fread`; za komunikaciju s korisnikom `printf` i `scanf`. Sve su to funkcije standardne C biblioteke.
- ▶ Svaka od njih je **funkcija omotač** (*wrapper*) — u konačnici se svodi na jedan ili više **sistemskih poziva**.
- ▶ Podsjetnik s prvog predavanja: sistemski pozivi jedino su sučelje prema jezgri.
- ▶ Zato ćemo raditi izravno sa sistemskim pozivima — da vidimo što se zapravo događa “ispod” funkcija koje ste dosad koristili.

Sve je datoteka

Isti mali skup sistemskih poziva koristi se za:

- ▶ komunikaciju s korisnikom preko terminala,
- ▶ rad s datotekama na disku,
- ▶ upravljanje uređajima,
- ▶ mrežnu komunikaciju (socketi),
- ▶ komunikaciju između procesa.

Zato ovo poglavlje ima daleko širu primjenu nego što mu naslov daje naslutiti — ono je temelj za razumijevanje UNIX logike.

Deskriptor datoteke

- ▶ Jezgra sve otvorene datoteke referencira **nenegativnim cijelim brojevima** — **deskriptorima datoteke** (*file descriptors*).
- ▶ Deskriptor je apstraktna referenca koju proces dobiva pri otvaranju datoteke i koristi za sve daljnje operacije: čitanje, pisanje, pozicioniranje, do zatvaranja.
- ▶ Proces **ne mora znati ništa** o datoteci iza deskriptora: gdje je fizički, radi li se o datoteci na disku, terminalu ili mrežnom portu.
- ▶ Jezgra o svakom deskriptoru čuva podatke i sama bira odgovarajuću komunikacijsku rutinu. Sučelje je za programera jedinstveno.

Predefinirani deskriptori

Pri pokretanju programa deskriptori 0, 1 i 2 u pravilu su već zauzeti:

Deskriptor	Naziv	Izvor / odredište
0	standardni ulaz	tipkovnica
1	standardni izlaz	korisnički terminal
2	standardni izlaz za greške	korisnički terminal

- ▶ Konvencija datira iz ranih 1970-ih i danas je dio POSIX standarda.
- ▶ U `<unistd.h>` definirane su simboličke konstante `STDIN_FILENO`, `STDOUT_FILENO` i `STDERR_FILENO` — koristite njih umjesto golih brojeva.
- ▶ Jezgra pri otvaranju uvijek dodjeljuje **najniži slobodan** deskriptor, pa `open()` u pravilu vraća 3 ili više.

2. **Sistemiški pozivi**

open()

Osnovna funkcija za otvaranje i, po potrebi, kreiranje datoteka.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open(const char *pathname, int oflag, /* mode_t mode */);
```

- ▶ **Povratna vrijednost:** deskriptor otvorene datoteke, ili -1 u slučaju greške.
- ▶ **pathname** — putanja do datoteke,
- ▶ **oflag** — kombinacija bitovnih konstanti koja određuje način otvaranja,
- ▶ **mode** — prava pristupa, navodi se samo pri kreiranju nove datoteke.

Zastavice: način pristupa

Argument oflag mora sadržavati **točno jednu** od tri konstante:

- ▶ `O_RDONLY` — otvori samo za čitanje,
- ▶ `O_WRONLY` — otvori samo za pisanje,
- ▶ `O_RDWR` — otvori za čitanje i pisanje.

Uz nju se bitovnim *OR*-om (`|`) mogu dodati opcionalne:

- ▶ `O_CREAT` — stvori datoteku ako ne postoji (traži treći argument),
- ▶ `O_APPEND` — prije svakog pisanja pomakni offset na kraj datoteke,
- ▶ `O_TRUNC` — obriši postojeći sadržaj,
- ▶ `O_EXCL` — uz `O_CREAT`: greška ako datoteka već postoji; provjera i kreiranje su **atomska operacija**,
- ▶ `O_NONBLOCK`, `O_SYNC` — neblokirajući rad, odnosno čekanje fizičkog upisa.

Tipične kombinacije

```
O_WRONLY | O_TRUNC      /* pisanje, obriši sadržaj */
O_WRONLY | O_CREAT      /* pisanje; stvori ako ne postoji */
O_WRONLY | O_CREAT | O_TRUNC /* pisanje ispočetka */
O_WRONLY | O_CREAT | O_EXCL /* stvori novu; greška ako postoji */
O_WRONLY | O_APPEND     /* pisanje na kraj datoteke */
```

Ovih pet kombinacija pokriva golemu većinu stvarnih slučajeva.

Prava nove datoteke

Treći argument mode navodi se uz `O_CREAT`. Zadaje se oktalno ili konstantama iz `<sys/stat.h>`:

Konstanta	Oktalno	Značenje
<code>S_IRUSR</code>	<code>0400</code>	čitanje za vlasnika
<code>S_IWUSR</code>	<code>0200</code>	pisanje za vlasnika
<code>S_IXUSR</code>	<code>0100</code>	izvršavanje za vlasnika
<code>S_IRGRP</code> , <code>S_IWGRP</code> , <code>S_IXGRP</code>	<code>0040</code> , <code>0020</code> , <code>0010</code>	isto za grupu
<code>S_IROTH</code> , <code>S_IWOTH</code> , <code>S_IXOTH</code>	<code>0004</code> , <code>0002</code> , <code>0001</code>	isto za ostale

`S_IRUSR` | `S_IWUSR` | `S_IRGRP` | `S_IROTH` ekvivalentno je zapisu `0644`.

creat()

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int creat(const char *pathname, mode_t mode);
```

Ekvivalentno je pozivu:

```
open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Iz toga slijede dva ograničenja: datoteka se otvara **samo za pisanje**, a postojeći sadržaj se briše. U modernom kodu obično se ide izravno preko `open()`-a.

Zašto creat bez e?

- ▶ Nije pogreška — naslijeđeno je iz prvih verzija UNIX-a s početka 1970-ih, kad je ime sistemskog poziva smjelo imati najviše šest znakova.
- ▶ Jedna od najpoznatijih neobičnosti UNIX-a.

Na pitanje što bi promijenio u UNIX-u kad bi mogao, **Ken Thompson** je odgovorio: *“I’d spell creat with an e.”*

close()

```
#include <unistd.h>
```

```
int close(int filedes);
```

- ▶ Zatvara datoteku otvorenu na deskriptoru `filedes` i oslobađa deskriptor za ponovnu upotrebu.
- ▶ Vraća 0 u slučaju uspjeha, -1 u slučaju greške.
- ▶ Kad proces završi, sve otvorene datoteke zatvaraju se **automatski**, pa eksplicitni poziv nije strogo nužan.
- ▶ Ipak ga koristite: broj deskriptora po procesu je ograničen, a kod nekih tipova datoteka (npr. socketa) zatvaranje pokreće važne završne operacije.

read()

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```

- ▶ **Povratna vrijednost:** broj stvarno pročitanih bajtova, 0 na kraju datoteke (*EOF*), -1 u slučaju greške.
- ▶ **buff** — pokazivač na memoriju u koju se upisuje. `read()` **ne alocira** memoriju; to je posao programera.
- ▶ **nbytes** — najviše koliko bajtova pokušati pročitati.
- ▶ Čitanje kreće od trenutnog **file offseta**, koji je nakon otvaranja 0 i pomiče se nakon svakog čitanja.
- ▶ Pročitano može biti **manje** od traženog: kraj datoteke, čitanje s terminala ili s mrežnog socketa.

write()

```
#include <unistd.h>
```

```
ssize_t write(int filedes, const void *buff, size_t nbytes);
```

- ▶ **Povratna vrijednost:** broj stvarno upisanih bajtova, ili -1 u slučaju greške.
- ▶ Za obične datoteke POSIX jamči da je u uspješnom slučaju jednak `nbytes`; kod socketa može biti manji.
- ▶ Pisanje kreće od trenutnog offseta, koji se nakon upisa pomiče.
- ▶ Iznimka je `O_APPEND`: tada jezgra prije svakog pisanja postavlja offset na kraj datoteke.

`ssize_t` (*signed size*) POSIX-ov je ekvivalent tipa `size_t` koji može biti negativan — upravo zato da funkcija može vratiti -1.

Obrada grešaka

- ▶ Sistemski pozivi grešku signaliziraju povratnom vrijednošću -1, a **razlog** upisuju u globalnu varijablu `errno`.
- ▶ `perror("open")` ispisuje na `stderr` zadani tekst i tekstualni opis zadnje greške.

```
fd = open("moja_datoteka.txt", O_RDONLY);  
if (fd == -1) {  
    perror("open");  
    return 1;  
}
```

Povratnu vrijednost provjeravajte **uvijek**. Program koji nastavi raditi s deskriptorom -1 ponaša se nepredvidivo.

3. **Primjeri**

read_file.c

```
int main() {
    int n, fd;
    char s;

    fd = open("moja_datoteka.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    while ((n = read(fd, &s, 1)) > 0)
        write(STDOUT_FILENO, &s, 1);

    close(fd);
    return 0;
}
```

read_file.c — što program radi

- ▶ Otvara datoteku samo za čitanje, čita je **znak po znak** i svaki znak ispisuje na standardni izlaz.
- ▶ Ilustrira osnovni slijed rada s datotekom: `open` → `read / write` → `close`.
- ▶ Uočite `STDOUT_FILENO` — pišemo na deskriptor 1, isti onaj koji smo prošli put preusmjeravali operatorom `>`.
- ▶ Petlja završava kad `read()` vrati 0, dakle na kraju datoteke.

Nedostatak: jedan sistemski poziv po znaku. Za datoteku od megabajta to je dva milijuna prijelaza u jezgru.

io_copy.c

```
#define BUFFSIZE 1024

int main() {
    int n;
    char buf[BUFFSIZE];

    while ((n = read(STDIN_FILENO, buf, BUFFSIZE)) > 0)
        if (write(STDOUT_FILENO, buf, n) != n) {
            perror("write");
            return 1;
        }

    if (n < 0)
        perror("read");

    return 0;
}
```


io_copy.c — što program radi

- ▶ Kopira standardni ulaz na standardni izlaz, čitajući u **međuspremnik** veličine `BUFSIZE`.
- ▶ Time se broj sistemskih poziva smanjuje višestruko u odnosu na čitanje znak po znak.
- ▶ Uočite da se u `write()` prosljeđuje `n`, a **ne** `BUFSIZE`: zadnje čitanje gotovo nikad nije puno.
- ▶ Primjer je ujedno ilustracija načela “sve je datoteka”: bez preusmjeravanja ulaz je tipkovnica, izlaz terminal, a kôd je isti kao za datoteke na disku.

```
$ ./io_copy
Prvi red teksta
Prvi red teksta
^D
```

io_copy.c uz preusmjerenje

```
$ ./io_copy > datoteka.txt
```

```
Prvi red teksta
```

```
Drugi red teksta
```

```
^D
```

```
$ cat datoteka.txt
```

```
Prvi red teksta
```

```
Drugi red teksta
```

- ▶ Isti program, bez ijedne izmjene u kodu, postaje jednostavan alat za upis teksta u datoteku.
- ▶ Program ne zna da je preusmjeren — za njega je to i dalje deskriptor 1.
- ▶ Ctrl+D je oznaka kraja ulaza (EOF): `read()` vrati 0 i petlja završava.

Argumenti naredbenog retka

- ▶ Dosadašnji primjeri imaju ime datoteke **ugrađeno u izvorni kôd**. Za svaku drugu datoteku trebalo bi mijenjati kôd i iznova prevoditi.
- ▶ Rješenje su **argumenti naredbenog retka** — tokeni koje korisnik upiše iza imena naredbe, a ljuska ih prosljeđuje programu (cp `izvor.txt` `cilj.txt`).

```
int main(int argc, char *argv[])
```

- ▶ `argc` — broj argumenata, **uključujući ime programa**,
- ▶ `argv[0]` — ime kojim je program pokrenut, `argv[1]` nadalje — stvarni argumenti.

Za poziv `./f_write izlaz.txt`: `argc = 2`, `argv[0] = "./f_write"`, `argv[1] = "izlaz.txt"`.

f_write.c

```
#define FMODE S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH

int main(int argc, char *argv[]) {
    int fd, n; char s;
    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 0;
    }
    fd = creat(argv[1], FMODE);
    if (fd == -1) {
        perror("creat");
        return 1;
    }
    while ((n = read(STDIN_FILENO, &s, 1)) > 0)
        write(fd, &s, 1);
    close(fd);
    return 0;
}
```

f_write.c — provjera argumenata

- ▶ Provjera `argc != 2` na samom početku uobičajen je uzorak u svim UNIX programima.
- ▶ U poruci se koristi `argv[0]`, pa uputa uvijek prikazuje **stvarno ime** pod kojim je program pozvan — i ako je preimenovan ili pozvan preko simboličke veze.

```
$ ./f_write izlaz.txt
Prvi red teksta
^D
$ cat izlaz.txt
Prvi red teksta
```

Demo: sistemski pozivi na djelu

Cilj: pokazati da naši programi rade jednako kao standardni UNIX alati.

```
./io_copy < /etc/hostname  
./f_write test.txt  
ls -l test.txt  
strace -c ./read_file
```

► `strace -c` prebrojava sistemske pozive — usporedite `read_file` i `io_copy`.

Poanta: `cat` nije ništa posebno; to je program s `open`, `read`, `write` i `close`.

4.

Pozicioniranje i prava

lseek()

```
#include <sys/types.h>
#include <unistd.h>
```

```
off_t lseek(int filedes, off_t offset, int whence);
```

- ▶ Mijenja **file offset** otvorene datoteke; koristimo ga kad želimo čitati ili pisati na određenoj poziciji bez sekvencijalnog prolaska.
- ▶ **Povratna vrijednost**: novi offset od početka datoteke, ili (off_t) - 1 u slučaju greške.
- ▶ **whence** — referentna točka:
 - ▶ SEEK_SET — od početka datoteke,
 - ▶ SEEK_CUR — od trenutnog offseta,
 - ▶ SEEK_END — od kraja datoteke (pomak može biti i negativan).

Pomicanje je samo **logičko**, na razini jezgrinog offseta. Disk se dira tek pri sljedećem read()-u ili write()-u.

f_strip.c

```
int main() {
    char buf1[] = "Prvi redak teksta";
    char buf2[] = "Drugi redak teksta";
    int fd;

    fd = open("file.strip", O_WRONLY | O_CREAT | O_TRUNC, (mode_t)0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    write(fd, buf1, strlen(buf1)+1);
    lseek(fd, 15, SEEK_SET);
    write(fd, buf2, strlen(buf2)+1);
    close(fd);
    exit(0);
}
```

Provjere povratnih vrijednosti izostavljene su radi preglednosti.

f_strip.c — rezultat

```
$ ./f_strip
```

```
$ cat file.strip
```

Prvi redak teksDrugi redak teksta

- ▶ Prvih 15 bajtova (Prvi redak teks) ostalo je iz prvog upisa.
- ▶ Od 15. bajta nadalje drugi upis je **prepisao** postojeći sadržaj.
- ▶ Offset koji jezgra pamti nema veze s fizičkim rasporedom blokova na disku — pozicioniranje se slobodno kombinira s čitanjem i pisanjem.

Prava pristupa i maska

- ▶ Prava koja tražimo argumentom mode **nisu nužno ona koja ćemo dobiti.**
- ▶ Svaki proces ima **masku kreiranja datoteka** (umask) — bitove koji se iz traženih prava **uklanjaju.**

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
mode_t umask(mode_t cmask);
```

- ▶ Rezultantna prava su $\text{mode} \& \sim \text{cmask}$. Poziv vraća **prethodnu** vrijednost maske.
- ▶ Maska se nasljeđuje od roditeljskog procesa; tipična vrijednost u ljusci je 0022 — oduzima pravo pisanja grupi i ostalima.

perm_mask.c

```
#define PRAVA S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH

int main() {
    umask(0);
    if (creat("datoteka1", PRAVA) < 0)
        perror("creat datoteka1");

    umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
    if (creat("datoteka2", PRAVA) < 0)
        perror("creat datoteka2");

    return 0;
}
```

Dvije datoteke, **ista** zatražena prava (rw-rw-r--), različite maske.

perm_mask.c — rezultat

```
$ ./perm_mask
$ ls -al datoteka1 datoteka2
-rw-rw-r-- 1 dkrst users 0 Jan 16 16:23 datoteka1
-rw----- 1 dkrst users 0 Jan 16 16:23 datoteka2
```

- ▶ datoteka1 — maska je 0, pa su zadržana sva zatražena prava.
- ▶ datoteka2 — maska isključuje prava grupi i ostalima, iako ih je program tražio.

Isto vrijedi i za datoteke koje stvarate u ljusci: naredba `umask` bez argumenta ispisuje trenutnu vrijednost maske.

Što smo naučili

- ▶ S datotekom radimo samo pet stvari: otvaranje, čitanje, pisanje, pozicioniranje i zatvaranje.
- ▶ **Deskriptor** je nenegativan cijeli broj kojim proces referencira otvorenu datoteku; 0, 1 i 2 su zauzeti pri pokretanju.
- ▶ `open()` vraća najniži slobodan deskriptor; način otvaranja zadaje se kombinacijom zastavica.
- ▶ `read()` vraća broj pročitanih bajtova, 0 na kraju datoteke, -1 kod greške — povratnu vrijednost provjeravamo uvijek.
- ▶ Čitanje u međuspremnik višestruko smanjuje broj sistemskih poziva.
- ▶ `lseek()` mijenja file offset; pomak je logički, disk se dira tek pri sljedećem pristupu.
- ▶ Stvarna prava nove datoteke su `mode & ~cmask` — maska oduzima ono što je program tražio.

Strukture podataka i dijeljenje datoteka

- ▶ tablica procesa, tablica datoteka i v-node tablica,
- ▶ dijeljenje datoteka između procesa,
- ▶ race condition i atomske operacije,
- ▶ `dup()` i `dup2()`,
- ▶ sistemski pozivi naspram funkcija standardne C biblioteke.

Do tada: prevedite i pokrenite primjere iz poglavlja 3.